

# Development Guide

---

## Purpose

---

This guide is for developers/agents extending the AI Inference Server Management Platform. It explains conventions, implementation priorities, and how to work safely in this codebase.

## 1. Working principles

---

- Keep everything container-first
- Prefer explicit, traceable state transitions in DB
- Treat logs/artifacts as product features, not debug leftovers
- Keep provider-specific logic isolated behind adapters
- Keep playbooks Git-centric for collaboration and reproducibility

## 2. Implementation roadmap

---

### Phase 1 (foundation)

- [x] Monorepo scaffolding
- [x] Compose stack with backend/frontend/worker/postgres/redis/litellm
- [x] Initial DB schema + Alembic
- [x] Base docs and env templates

### Phase 2 (core orchestration)

- [ ] Clore.ai API client (rent/list/terminate)
- [ ] Server registration and capability extraction
- [ ] SSH connectivity manager + credential handling
- [ ] Playbook catalog + git-sync
- [ ] Celery task pipeline for deploy workflows

### Phase 3 (runtime and observability)

- [ ] Tunnel manager and endpoint tracking
- [ ] LiteLLM route management integration
- [ ] Rich run logs and downloadable artifacts
- [ ] UI pages for operations + failure diagnostics

### Phase 4 (intelligence)

- [ ] Compatibility scoring (playbook × GPU × model)
- [ ] Success/failure analytics dashboard
- [ ] Optional semantic search on logs

## 3. Coding conventions

---

### Backend (Python)

- Python 3.11
- Type hints required in new modules
- Pydantic for input/output schemas
- SQLAlchemy 2.0 style where possible
- Keep long workflows in Celery tasks, not request handlers

### Frontend (Next.js)

- TypeScript-first
- App Router structure ( `src/app` )
- Keep API clients in dedicated service modules
- Build presentational components separately from data-fetching logic

## 4. Database and migrations

---

- All schema changes must use Alembic revision files
- Do not modify existing applied migrations in-place
- Keep migration descriptions specific and auditable

Suggested pattern:

1. Edit SQLAlchemy models
2. Generate migration ( `alembic revision --autogenerate -m "..."` )
3. Review generated SQL carefully
4. Apply with `alembic upgrade head`

## 5. Celery task design

---

- Make tasks idempotent where possible
- Persist `task_runs` row at start and update through lifecycle
- On failure, record exception and artifact pointers
- Avoid long blocking loops without heartbeats/checkpoints

## 6. Logging standards

---

Every orchestration task should log:

- provider account / server identifiers
- playbook identity + git commit
- command start/end timestamps
- stdout/stderr artifact paths
- normalized outcome ( `PENDING` , `RUNNING` , `SUCCESS` , `FAILED` , `PARTIAL` )

## 7. Environment management

---

- Root `.env` configures Compose-level defaults
- `backend/.env` and `frontend/.env.local` support service-specific overrides
- Never commit real secrets

## 8. Local development flow

---

```
docker compose up --build
# backend migrations
docker compose exec backend alembic upgrade head
```

## 9. Quality gates (recommended)

---

Backend:

- `ruff` / `black` / `mypy` / `pytest`

Frontend:

- `eslint` / `prettier` / `tsc`

Add CI once repository workflows are initialized.

## 10. Architecture decision records (ADR)

---

When introducing major behavior changes (provider abstraction, tunnel design, log storage format), add a short ADR under `docs/adr/` with:

- context
- decision
- consequences

This helps future agents understand why choices were made.